

If semantic knowledge, or knowledge that's known by the harness regarding the request. If all of this knowledge is exposed to the backend inference engine, what benefits would it be able to provide this backend?

If the harness exposes its knowledge to the inference backend, the backend can stop treating every request as an anonymous collection of tokens.

It can make decisions based on:

- Why the request matters.
- When it will become ready.
- How quickly it must respond.
- How much work it will probably require.
- Whether its KV cache will probably be reused.
- Whether the work is optional, speculative, or on the critical path.

The main benefit is:

> The inference engine can prepare for future work and allocate resources according to the value of the work, instead of reacting only after requests arrive.

What the harness knows that SGLang normally does not

The harness may know:

Harness knowledgeExample	
-----	-----
Workflow phase	Waiting for a database tool
User status	A customer is actively waiting
Deadline	First token required within 100 ms
Expected readiness	Tool likely to finish in 30 seconds
Timing uncertainty	Tool may finish 2 seconds early or late
Business priority	Customer interaction is more important than reporting
Expected output	Usually 150 tokens; rarely more than 400
Cache reuse	The agent will almost certainly reuse its 80,000-token context
Recomputation cost	Losing the KV would require reprocessing 80,000 tokens
Optionality	A speculative research branch can be cancelled
Dependency structure	Three downstream tasks are waiting for this answer
Future workflow	After this tool result, the agent will perform a short decode

SGLang normally sees tokenized requests, cache matches, `max\_new\_tokens`, priority and current GPU state. It does not automatically know the broader workflow surrounding the request.

1. Better deadline scheduling

Suppose these requests are waiting:

- Customer-support agent: user waiting, 100 ms response target.
- Coding agent: large repository prefill, no immediate deadline.
- Reporting agent: background report due in ten minutes.

Without harness knowledge, SGLang might see only:

```
...
...

Request A: 200 uncached tokens
Request B: 80,000 uncached tokens
Request C: 5,000 uncached tokens
...
```

With harness knowledge, it sees:

```
...
...

Request A:
  user_waiting = true
  deadline_ms = 100
```

```
critical_path = true
```

Request B:

```
user_waiting = false
deadline_ms = none
phase = background_prefill
```

Request C:

```
deadline_ms = 600000
critical_path = false
...`
```

It can then:

- Schedule A first.
- Break B's prefill into smaller chunks.
- Delay C temporarily.
- Preserve a small amount of capacity for A's decode.

Benefit:

> The most important requests receive predictable latency, instead of merely sharing the GPU fairly.

This is more powerful than a static priority because deadline urgency changes with time. A low-priority request whose deadline is approaching may eventually become urgent.

2. Proactive KV-cache management

Consider an agent with an 80,000-token history that starts a 30-second database call.

The harness knows:

```
...
...
...
phase = tool_wait
expected_return = 30 seconds
reuse_probability = 0.98
recompute_tokens = 80,000
...`
```

The backend can reason:

- The KV is valuable because recomputation is expensive.
- It is not needed for approximately 30 seconds.
- Keeping all of it in scarce HBM would waste GPU memory.
- Moving it to distant storage may make restoration too slow.

It can therefore:

1. Move the inactive KV from HBM to host memory.
2. Use the freed HBM for active requests.
3. Retain metadata identifying the session and prefix.
4. Prefetch the KV shortly before the tool is expected to finish.
5. Immediately accelerate restoration if the harness sends `tool\_completed`.

Benefit:

> More active agents fit on the GPU without forcing expensive prompt recomputation when sleeping agents resume.

Without the harness hint, SGLang sees only that the KV is currently inactive. It cannot distinguish:

- An agent returning in 20 milliseconds.
- An agent returning in 30 seconds.
- An agent that will never return.

Those three caches should not be treated identically.

3. Better decisions about which cache to evict

Suppose HBM is full and SGLang must choose between two cached sessions:

Session	KV size	Reuse probability	Recomputation cost	Expected return			
-----	-----	-----	-----	-----	---	-----	---
Support agent	50 ms			80,000 tokens	98%	Very high	
Completed research branch	Probably never			100,000 tokens	5%	Medium	

A normal LRU policy may focus mainly on when each cache was last accessed.

A harness-aware policy can consider:

```

 $\text{cache value}
\approx
\text{reuse probability}
\times
\text{recomputation cost}
\times
\text{urgency}$ 

```

It would protect the support session and evict the completed research branch.

Benefit:

> KV blocks are retained according to their expected future value, not merely their recent access time.

4. Just-in-time GPU headroom

Permanently reserving 20% of the GPU for urgent requests would provide good latency but waste capacity when no urgent work exists.

Harness knowledge allows temporary reservation.

For example:

```

...
...

...
Tool expected to finish in 30 seconds
KV restoration requires 8 ms
Scheduling/setup requires 4 ms
Deadline after completion is 100 ms
Timing uncertainty is 2 seconds
...

```

The backend can continue using nearly the entire GPU during most of the tool wait.

Near the expected return, it can:

- Reduce the size of background prefill batches.
- Stop admitting additional long background work.
- Create a small amount of decode headroom.
- Prefetch the required KV.

If the tool is delayed, the harness updates the ETA and SGLang releases the reserved capacity.

Benefit:

> The system protects urgent work only when necessary, instead of leaving expensive GPU capacity permanently idle.

5. Smarter prefill chunking

Chunk size controls a tradeoff:

- Large prefill chunks improve throughput.
- Small chunks create more opportunities for urgent requests to enter.

Without future-work knowledge, SGLang must choose a relatively static compromise.

With harness knowledge:

```

'''
'''

```

```

'''

```

No urgent agent expected soon
→ use larger prefill chunks

High-priority tool expected to finish soon
→ use smaller prefill chunks

Tool completion event received
→ stop background prefill at the next boundary
'''

Concrete example:

```

'''
'''

```

```

'''

```

Normal period:
coding prefill chunk = 8,192 tokens

Support agent expected soon:
coding prefill chunk = 1,024 tokens

Support agent becomes ready:
schedule support prefill/decode first
'''

Benefit:

> Background work remains efficient during quiet periods but becomes easier to interrupt when important work is approaching.

6. More accurate memory admission

SGLang currently uses information such as `max\_new\_tokens` to estimate future KV growth. But `max\_new\_tokens` is only an upper limit.

For example:

```

'''
'''

```

```

'''

```

```

max_new_tokens = 8,000
historical output P50 = 120
historical output P95 = 350
'''

```

If SGLang reserves KV for all 8,000 tokens, it may underutilize HBM. If it assumes only 120 tokens, it may over-admit requests and later need to retract them.

The harness could provide:

```

'''
'''

```

```

'''

```

```

{
  "output_tokens_p50": 120,
  "output_tokens_p95": 350,

```

```
"output_tokens_max": 8000,  
"confidence": 0.91  
}  
...
```

The inference engine can reserve memory according to the service requirement:

- High-priority request: reserve closer to P95.
- Background request: reserve closer to P50 and allow retraction.
- Hard-deadline request: use a more conservative estimate.
- Optional speculative request: admit aggressively because it can be cancelled.

Benefit:

> Higher concurrency with fewer KV-memory failures and fewer request retractions.

7. Critical-path scheduling

Priority alone says that a request is important. Critical-path information says that other work is blocked by it.

Consider two equally important requests:

- Request A is an independent report.
- Request B must finish before five agent branches can continue.

The harness can mark:

```
...  
...  
  
...  
Request A:  
downstream_dependents = 0  
  
Request B:  
downstream_dependents = 5  
critical_path = true  
...
```

SGLang can favor B because completing it unlocks more useful work.

Benefit:

> The backend optimizes complete agent-task latency, not merely individual request latency.

This is particularly valuable in workflows containing:

- Planner calls.
- Tool-selection calls.
- Branch joins.
- Verification steps.
- Final synthesis requests.

8. Cancellation of low-value work

The harness may know that some work is optional:

```
...  
...  
  
...  
branch_type = speculative  
cancelable = true  
value_if_late = zero  
...
```

If GPU pressure rises, SGLang or the controller can:

- Pause the branch.
- Lower its priority.

- Release its KV.
- Cancel it completely.
- Avoid admitting its next prefill.

Concrete example:

Three research branches are exploring possible answers. Branch A finds decisive evidence. The harness knows branches B and C are no longer necessary and cancels them.

Benefit:

> The GPU stops performing work whose result will never be used.

This can produce greater savings than merely scheduling the same unnecessary work more efficiently.

9. Better batching

Semantic or workflow hints can identify requests that are suitable to run together.

Examples include:

- Same LoRA adapter.
- Same shared system prompt.
- Same agent template.
- Similar expected output lengths.
- Similar latency class.
- Same routing or tenant key.
- Same reusable prefix.
- Background requests with flexible completion times.

The backend could avoid mixing:

- A hard-deadline, short decode with a disruptive long prefill.
- A request expected to produce ten tokens with requests expected to produce thousands.
- High-priority interactive work with easily cancelable speculation.

Benefit:

> Batches can be formed for both efficiency and service quality.

10. Better model and execution-path selection

If multiple models or execution modes are available, harness knowledge could help choose among them.

For example:

...
...

...

Simple tool-result classification
→ small model

Customer-facing final response
→ large model

Low-risk draft
→ speculative decoding

Strict structured result
→ grammar-constrained decoding

Optional background summary
→ lower-cost model or delayed execution
...

This decision may sit above an individual SGLang worker, but the backend can expose availability and cost information to the controller.

Benefit:

> Expensive model execution is used only where it provides meaningful value.

11. Predictive rather than reactive behavior

This is the biggest architectural change.

Reactive backend

```\n```\n

```\n

Request arrives

- discover KV is missing
 - find memory
 - restore or recompute KV
 - schedule request
 - begin generation
- ```\n

Predictive backend

```\n```\n

```\n

Harness announces likely future request

- backend protects or restores KV
 - adjusts background chunking
 - prepares capacity
 - request becomes ready
 - generation begins quickly
- ```\n

The harness supplies knowledge of the future. SGLang supplies knowledge of current GPU conditions. Together, they can make better decisions than either system alone.

Complete example

Assume three agent tasks share one GPU.

| AgentHarness knowledgeBackend action | | |
|--|--|--|
| ----- ----- | | |
| Customer support | | User waiting; tool returning soon; 100 ms target; 98% KV reuse |
| Prefetch KV, reduce background chunk size and schedule first | | |
| Coding agent | | Large prefill; no immediate deadline; valuable background work |
| Run large chunks during quiet periods and smaller chunks near the support resume | | |
| Reporting agent | | Waiting 30 seconds; large KV; low urgency |
| Move KV to host memory and use its HBM for active work | | |

The resulting timeline is:

```\n```\n

```\n

During the tool wait:

- Coding agent uses most GPU compute.
- Reporting KV leaves HBM.
- Customer-support KV is temporarily stored in host memory.

Near expected tool completion:

- Customer-support KV is prefetched.
- Coding prefill chunks become smaller.
- Small decode headroom is created.

When the tool completes:

- Support request enters at high priority.
- Its KV is already available.

- It starts quickly.
- Background coding continues with remaining capacity.

After the support response:

- Temporary reservation is released.
- Coding returns to larger, efficient prefill chunks.

The benefits are:

- Lower tool-completion-to-first-token latency.
- More priority requests meeting their deadlines.
- More concurrent agent sessions per GPU.
- Less prompt recomputation.
- Fewer KV-related retractions.
- Better background throughput.
- Less permanently reserved GPU capacity.
- Lower cost per completed agent task.

The most useful signals

The harness should not simply send a label such as:

```

...
...

request_type = customer_support
...
```

That label leaves too much interpretation to SGLang.

It should send actionable information:

```

...
...

{
  "priority": 10,
  "user_waiting": true,
  "critical_path": true,
  "phase": "tool_wait",
  "next_ready_eta_ms": 30000,
  "eta_uncertainty_ms": 2000,
  "deadline_after_ready_ms": 100,
  "deadline_class": "hard",
  "output_tokens_p50": 150,
  "output_tokens_p95": 400,
  "reuse_probability": 0.98,
  "recompute_tokens": 80000,
  "cancelable": false,
  "preferred_cache_tier": "host"
}
...
```

This gives the backend facts it can convert into concrete resource decisions.

Important limitations

These hints cannot make impossible guarantees.

The backend still cannot:

- Meet every deadline during unlimited overload.
- Create compute when the GPU is fully saturated.
- Prefetch KV faster than PCIe or memory bandwidth allows.
- Predict exact output length perfectly.
- Rely safely on inaccurate or untrusted frontend hints.
- Interrupt every GPU operation instantaneously.

Therefore, hints should include uncertainty and confidence, and the inference engine should treat them as guidance constrained by actual measurements.

Hard deadlines additionally require:

- Admission control.
- Capacity reservation.
- Overload rejection or degradation.
- Fairness and tenant limits.
- Protection against clients falsely claiming high priority.

Recommended responsibility boundary

The harness should decide:

- Why the work matters.
- When it will probably become ready.
- Whether a user or downstream task is waiting.
- Whether the work is optional.
- How expensive losing its state would be.

SGLang should decide:

- Whether the request fits.
- Which requests can share a batch.
- Which KV blocks are physically available.
- Which cache transfer can be performed now.
- Whether preemption is safe.
- Which exact GPU batch should execute next.

So the core opportunity is:

> The harness explains the request's intent and future workflow. The inference engine combines that knowledge with live GPU and cache conditions to schedule, prepare and execute the work more intelligently.